

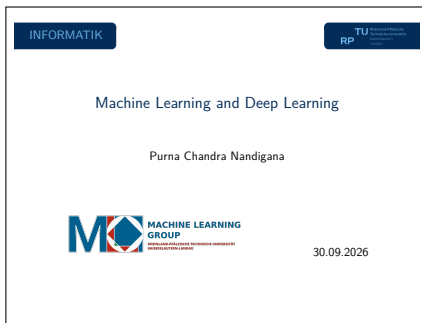
Presentation Script

Machine Learning and Deep Learning — Final Presentation

Purna Chandra Nandigana

How to use this document. Each section shows one slide and what to say for it, written in the first person. It is a guide, not something to read out word for word: the bold words are the points to hit. The total talk is about 15–17 minutes. All numbers come from the final project report.

Slide 1: Title

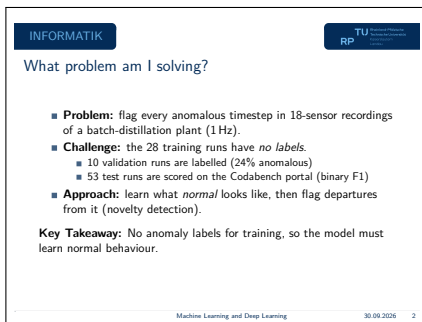


Suggested time: 20 s

Good morning. My name is Purna Chandra Nandigana. To-day I will present my project for Machine Learning and Deep Learning: **unsupervised anomaly detection** in sensor data from an industrial batch-distillation plant.

I will go through the problem, then the **two phases** of my work, the final result of **F1 = 0.63** on the competition portal, and what I learned about how reliable that result is.

Slide 2: What problem am I solving?



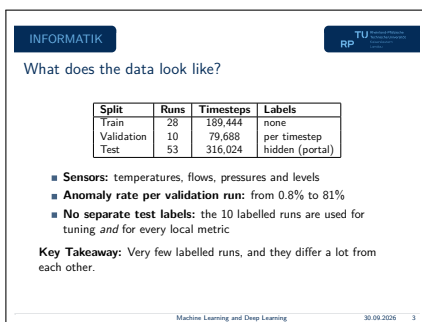
Suggested time: 1 min

The task is to decide, **for every single timestep**, whether the plant is behaving normally or not. There are 18 sensors, recorded once per second.

The hard part is the labels. The **28 training runs have no labels at all**. Only 10 validation runs tell me which timesteps are anomalous, and the 53 test runs are scored externally on Codabench with binary F1.

So I cannot train a normal classifier, because there are no anomalies to learn from. Instead I use **novelty detection**: the model learns what normal behaviour looks like, and anything that departs from it is flagged.

Slide 3: What does the data look like?



Suggested time: 50 s

Here are the three splits. Training is about 190,000 timesteps, validation about 80,000, and test about 316,000.

Two points matter for everything that follows. First, the anomaly rate **varies enormously between runs**: one validation run is under 1% anomalous, another is 81%. Second, there is **no separate labelled test set**. The same 10 runs are used to tune the model and to measure it. I will come back to what that means at the end.

Slide 4: Phase 1: Where did I start?

INFORMATIK

TU Braunschweig

Phase 1: Where did I start?

- **Baseline:** Isolation Forest on 216 tabular features
→ val. F1 = 0.437
- **First deep model:** hybrid of a shallow CNN autoencoder and a window Isolation Forest (weights 0.85 / 0.15)
- Its reported F1 $\approx$ 0.62 was measured on **window-level labels** and could not be reproduced

Key Takeaway: The starting point was a hybrid model whose numbers could not yet be trusted.

Machine Learning and Deep Learning30.08.20264

Suggested time: 1 min

Phase 1 started with two things. The first baseline was an **Isolation Forest** on 216 hand-built tabular features. It reached a validation F1 of **0.437**.

The first deep model was a **hybrid**, shown on the right: a shallow CNN autoencoder and a window-based Isolation Forest, fused with fixed weights of 0.85 and 0.15.

This hybrid had a reported F1 of about 0.62, but that number was measured on **window-level labels** and I could not reproduce it from any saved output. So I treat it as historical, not as a real baseline.

Slide 5: Phase 1: What did I have to fix first?

INFORMATIK

TU Braunschweig

Phase 1: What did I have to fix first?

- **Six correctness bugs** found while rebuilding the notebooks into one pipeline:
 - windows crossed run boundaries
 - threshold taken off by one
 - Isolation Forest features flattened wrongly
 - scoring per window instead of per timestep
 - training reverted from 50 to 10 epochs
 - fusion weight hand-set instead of tuned
- **Result after fixing:** hybrid val. F1 = 0.494
- **Portal:** 0.399 (public) / 0.471 (private)

Key Takeaway: Correct evaluation first; only then do the numbers mean anything.

Machine Learning and Deep Learning30.08.20265

Suggested time: 1 min 10 s

When I rebuilt the notebooks into one clean pipeline, I found **six correctness bugs**. For example, windows crossed the boundary between two different runs, the threshold was taken from the wrong index, and scoring was done per window even though the task is per timestep. The training length had also silently dropped from 50 to 10 epochs.

After fixing all six, the hybrid reached a validation F1 of **0.494**, and on the portal it scored **0.399 public and 0.471 private**.

This is lower than the old 0.62, but it is the **first number I could trust**. Without this step, every later comparison would have been meaningless.

Slide 6: Phase 1: How did the autoencoder improve?

INFORMATIK

TU Braunschweig

Phase 1: How did the autoencoder improve?

Round	CNN F1	CNN ROC-AUC
1: corrected pipeline	0.408	0.574
2: checkpoint selection	0.458	0.614
3: dilated architecture	0.484	0.605

- **Checkpoint selection:** keep the best validation epoch, not the last
- **Dilated architecture:** wider view of time (61 timesteps)
- **Problem left:** ROC-AUC stayed near 0.6 and the threshold flagged up to 89% of timesteps

Key Takeaway: F1 went up, but ranking quality did not: the score itself was the bottleneck.

Machine Learning and Deep Learning30.08.20266

Suggested time: 1 min 10 s

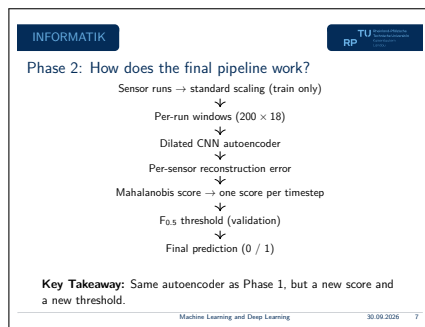
Next I improved the autoencoder itself in two rounds.

In **Round 2** I added checkpoint selection: training loss kept falling, but validation quality did not, so I keep the best epoch instead of the last one. F1 went from 0.408 to 0.458.

In **Round 3** I switched to a **dilated architecture**, so the model sees 61 timesteps of context. F1 rose to 0.484.

But look at the **ROC-AUC column**: it stayed around 0.6. ROC-AUC measures how well the score *ranks* anomalies, independent of any threshold. So the F1 gains came mostly from where the threshold sat, and the threshold was flagging up to 89% of timesteps. That told me the **score itself was the bottleneck**, which is what Phase 2 addresses.

Slide 7: Phase 2: How does the final pipeline work?

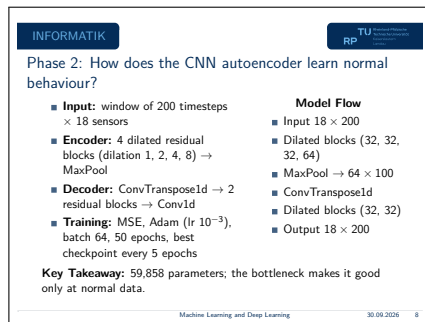


Suggested time: 1 min

This is the final pipeline, top to bottom.

The data is standardised using statistics from the training runs only. Each run is cut into windows of 200 timesteps by 18 sensors, and windows never cross from one run into another. The **dilated CNN autoencoder** reconstructs each window. The two new parts are in the middle and at the bottom: a **per-sensor Mahalanobis score** instead of a flat error, and an **F_{0.5} threshold** instead of F1. The network is the same as at the end of Phase 1; what changed is how its output becomes a decision.

Slide 8: Phase 2: How does the CNN autoencoder learn normal behaviour?



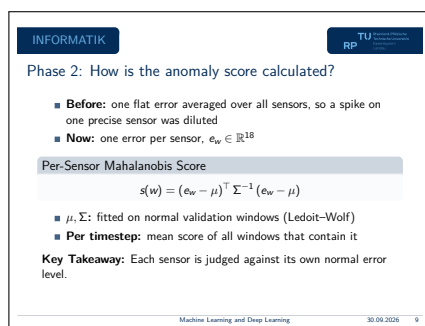
Suggested time: 1 min

The autoencoder takes a window of 200 timesteps and 18 sensors. The **encoder** has four dilated residual blocks with dilations 1, 2, 4 and 8, followed by max-pooling, which halves the time dimension. The **decoder** upsamples again and outputs a reconstruction of the same shape.

It is trained only on the unlabelled training windows, with mean squared error and Adam, for 50 epochs.

The key idea is the **bottleneck**: the model cannot just copy its input, so it learns to reconstruct *normal* patterns well. When something abnormal happens, the reconstruction is worse, and that error is the signal. The whole model has about **60,000 parameters**.

Slide 9: Phase 2: How is the anomaly score calculated?



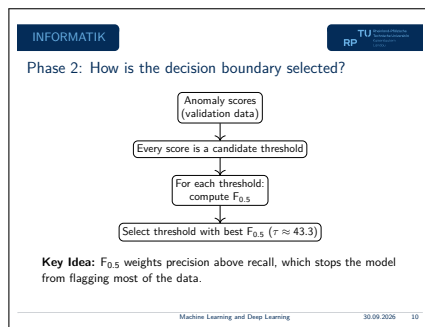
Suggested time: 1 min 20 s

This is the most important slide. Before, I averaged the reconstruction error over all 18 sensors into one number. The problem is that a **spike on one precise sensor gets diluted** by ordinary noise on the other 17.

Now I keep **one error value per sensor**, an 18-dimensional vector. I fit a Gaussian model of what this vector looks like in normal windows, and score each window by its **Mahalanobis distance** from that normal model. This weights each sensor by its own normal error level and accounts for correlations between sensors. The covariance is estimated with Ledoit-Wolf shrinkage for stability.

Finally, every timestep gets the average score of all windows that contain it.

Slide 10: Phase 2: How is the decision boundary selected?



Suggested time: 50 s

Slide 11: Phase 2: Did the new score help?

INFORMATIK RP TU/e

Phase 2: Did the new score help?

Metric (validation)	Round 3	Round 4
F1	0.484	0.581
Precision / recall	0.347 / 0.796	0.601 / 0.562
ROC-AUC	0.605	0.768
Flagged (true 0.24)	0.894	0.224

- **ROC-AUC** +0.16; better ranking, independent of the threshold
- **Portal:** 0.471 → **0.59**

Key Takeaway: Changing the score, not the network, gave the largest gain of the project.

Machine Learning and Deep Learning 30.09.2026 11

Suggested time: 1 min

Slide 12: Phase 2: Did fusion with LOF help?

INFORMATIK RP TU/e

Phase 2: Did fusion with LOF help?

- **Test:** fuse CNN with a Local Outlier Factor detector (contributed by my teammate)
- **Weight** chosen by 5-fold cross-validation grouped by run

Val. F1	CNN only	LOF only	Hybrid
raw features	0.592	0.461	0.549
standardised	0.568	0.481	0.568

- 9 of 10 folds chose the CNN alone (LOF weight 0.03 / 0.00)

Key Takeaway: Fusion was rejected; the final model is the CNN autoencoder alone.

Machine Learning and Deep Learning 30.09.2026 12

Suggested time: 1 min

To turn scores into 0 or 1, I use the validation data. Every score is a candidate threshold; for each one I compute the $F_{0.5}$ score, and I keep the best. The chosen threshold is about 43.3.

Why $F_{0.5}$ and not F1? Because $F_{0.5}$ **weights precision more than recall**. In Phase 1 the F1-optimal threshold flagged most of the data. $F_{0.5}$ keeps the model from simply calling everything an anomaly.

Yes, clearly. With the same network, F1 went from 0.484 to **0.581**, and more importantly **ROC-AUC went from 0.605 to 0.768**.

Because ROC-AUC does not depend on the threshold, this jump means the model is genuinely better at separating anomalies from normal data. The share of flagged timesteps also dropped from 89% to 22%, close to the true rate of 24%. One honest caveat: the scoring and the threshold changed in the same round, so the F1 gain mixes both. The ROC-AUC gain belongs to the scoring change alone. On the portal this round scored **0.59**.

I then asked whether a second detector could still add something. My teammate contributed a **Local Outlier Factor** detector on cross-sensor features, and I fused it with the CNN. To avoid fooling myself, the fusion weight was chosen with **5-fold cross-validation grouped by run**, so no run was used both to pick the weight and to evaluate it.

The result was clear: in **9 of 10 folds the search chose the CNN alone**, and the hybrid never beat it. So I rejected fusion, and the final model is the CNN autoencoder on its own.

Slide 13: Final Performance Comparison (Test Set)

Final Performance Comparison (Test Set)		
Model	Val. F1	Portal F1 (Test)
Phase 1: hybrid	0.494	0.399 / 0.471
Phase 2: Mahalanobis score	0.581	0.59
Phase 2: CNN only (final)	0.602	0.63

■ Final model: val. ROC-AUC 0.763
■ 0.59 → 0.63: a new training run of the same method (run-to-run spread about ± 0.02)

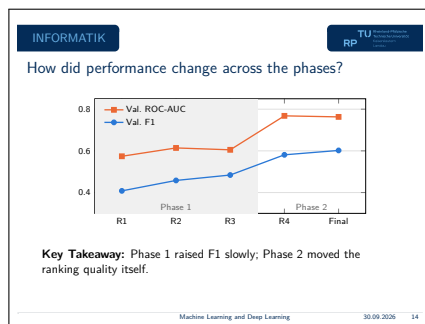
Key Insight: The CNN autoencoder with Mahalanobis scoring achieves the best result, portal F1 = 0.63.

Suggested time: 1 min

Here are the portal scores. The Phase 1 hybrid scored 0.399 and 0.471. The new Mahalanobis score brought it to 0.59. The **final model, the CNN autoencoder alone, scored 0.63**, with validation F1 0.602 and ROC-AUC 0.763.

I want to be precise about the last step. The move from 0.59 to 0.63 came from a **new training run of the same method**, not from a new idea. Identical runs differ by about ± 0.02 F1. The real improvement between the phases is the **new scoring method**.

Slide 14: How did performance change across the phases?



Suggested time: 40 s

This chart summarises the whole story. The blue line is validation F1, the orange line is ROC-AUC.

In **Phase 1**, on the grey side, F1 climbs slowly, but ROC-AUC stays flat around 0.6. In **Phase 2**, ROC-AUC jumps to about 0.77. That jump is the moment the model actually became better at ranking anomalies, rather than just having its threshold moved.

Slide 15: How reliable is the final result?

How reliable is the final result?	
■ Tested after the final submission (none adopted):	
■ F1-optimal threshold: ± 0.024 val. F1, but flags 84% of test	
■ Per-run normalisation: ROC-AUC 0.769 → 0.677	
■ Normal model fitted on train: ROC-AUC 0.611	
■ Leave-one-run-out: validation is optimistic by 0.070 ROC-AUC and 0.058 F1	
■ Val. vs test: 24% vs 62% of timesteps flagged at the same threshold	
Key Takeaway: The method held up, but validation numbers should be read as optimistic.	

Suggested time: 1 min 20 s

After the final submission I tested three more ideas on one shared network. An **F1-optimal threshold** gave $+0.024$ F1, which is within noise, and it would flag 84% of the test set. **Per-run normalisation** damaged the ranking. Fitting the normal model on **training data** was much worse. None was adopted.

The most important finding is about evaluation. My normal model is fitted on normal windows from the *same* validation runs it then scores. When I refit it on the other nine runs only, **ROC-AUC drops by 0.07 and F1 by 0.058**. So my validation numbers are **optimistic**.

I also see that the same threshold flags 24% of validation but 62% of test, so the test scores run higher.

Slide 16: What are the key parameters of the final system?

INFORMATIK RP TU/e

What are the key parameters of the final system?

Component	Parameter Value
Windowing	Window size = 200, train stride = 10
CNN Autoencoder	Dilations = 1, 2, 4, 8
CNN Autoencoder	Filters = 32, 32, 32, 64
CNN Autoencoder	Batch size = 64
CNN Autoencoder	Learning rate = 1e-3, 50 epochs
Scoring	Mahalanobis, Ledoit-Wolf covariance
Threshold	$F_{0.5}$, $\tau \approx 43.3$

Key Insight: Performance depended most on how the error is scored and thresholded.

Machine Learning and Deep Learning 30.09.2026 16

Suggested time: 30 s

For reference, these are the settings of the final model: windows of 200 with training stride 10, dilations 1 to 8, batch size 64, learning rate 10^{-3} for 50 epochs, Mahalanobis scoring with Ledoit–Wolf covariance, and an $F_{0.5}$ threshold of about 43.3.

The main message is that the settings that mattered most were **how the error is scored and how the threshold is chosen**, not the size of the network.

Slide 17: Summary & Next Steps

INFORMATIK RP TU/e

Summary & Next Steps

- **Summary:** dilated CNN autoencoder with per-sensor Mahalanobis scoring and an $F_{0.5}$ threshold.
- **Result:** portal F1 = **0.63** (val. F1 0.602, ROC-AUC 0.763).
- **Key Outcome:** Phase 2's new score lifted ROC-AUC from 0.605 to 0.768.
- **Next Steps:**
 - Leave-one-run-out validation for every tuning step
 - Confirm how the portal computes F1
 - Average over several training seeds

Final Message: How the model's error is scored mattered more than model size.

Machine Learning and Deep Learning 30.09.2026 17

Suggested time: 1 min

To summarise: my final model is a **dilated CNN autoencoder with per-sensor Mahalanobis scoring** and an $F_{0.5}$ threshold. It reached **F1 = 0.63 on the portal**.

The key step was Phase 2's new score, which raised ROC-AUC from 0.605 to 0.768.

Next, I would use **leave-one-run-out validation for every tuning step**, confirm exactly how the portal computes F1, and average several training seeds to reduce run-to-run noise. My final message: **how the model's error is scored mattered more than how big the model is**. Thank you, I am happy to take questions.

Possible Questions and Short Answers

Why not a supervised model? The training runs have no labels, and only 10 runs are labelled. Those are needed for tuning and evaluation, so there is nothing left to train a classifier on.

Why is the portal score (0.63) higher than validation (0.60)? The two are measured on different data, and the portal's exact F1 aggregation and the test anomaly rate are not known. I do not read the higher portal score as proof that the model generalises well; my leave-one-run-out test suggests validation is, if anything, optimistic.

Is 0.59 to 0.63 an improvement of the method? No. It is a new training run of the same method. Runs of the same configuration differ by about ± 0.02 F1.

Why does Mahalanobis scoring help? It judges each sensor against its own normal error level and accounts for correlations, so a spike on one precise sensor is not averaged away by noise on the others.

Why $F_{0.5}$ instead of F1? The F1-optimal threshold flagged most of the data. $F_{0.5}$ favours precision and keeps the flagged share close to the true anomaly rate.

What would you do next? Use leave-one-run-out validation for all tuning, confirm the portal metric, and average several training seeds.